

Homework 1: Relational Algebra and SQL

COSC 3337 — Databases — Fall 2026

Released	Wednesday, September 9
Part 0 due	Friday, September 12, 11:59 pm
Everything else due	Sunday, September 20, 11:59 pm
Total	100 points, plus 5 bonus

Before you start

This assignment has two halves that use the same database. Part 1 is written: you work relational algebra out on paper, by hand. Part 2 is code: you run SQL against MySQL on your own virtual machine. The last three SQL questions ask you to translate expressions you wrote in Part 1, so **do Part 1 first** — it is not busywork you can skip past to get to the terminal.

What to submit, in two files:

1. `hw1.pdf` — your answers to Part 0 and Part 1.
2. `hw1.sql` — your answers to Part 2, in the format described on page 8.

Typesetting bonus (5 points). Part 1 is full of symbols that are tedious to type. You may hand-write your answers and scan them — if you do, use a scanning app rather than a photograph, and label each problem clearly. But if you typeset Part 1 in L^AT_EX, you earn 5 bonus points. A template with every symbol you need already defined as a macro is posted alongside this handout; open it in Overleaf and you will not have to type a single backslash-sigma. Submit only the PDF either way.

How long this should take. Roughly five to seven hours, split about evenly between the two parts, plus twenty minutes for Part 0. It is not a sit-down-the-night-before assignment, mostly because Part 0 can go wrong in ways that take a day to fix.

Collaboration and late work are governed by the course syllabus. Read it if you have not.

Part 0 — Your database VM

(5 points)

Due Friday, September 12 — earlier than the rest, on purpose. If something is wrong with your VM, we both want to know about it with eight days left, not eight hours.

Each of you has your own virtual machine running Ubuntu with MySQL already installed. You have full administrative access to it. Nobody else uses it, so you cannot break anyone else's work — but you can break your own, and that is what most of this page is about.

Read this before you type anything

These VMs are built from a public image, and machines like them are scanned and attacked continuously. There is also **no backup image and no console access**: if you lock yourself out or damage the operating system, the only repair available is deleting your VM and creating a new one, which you must email me to do.

So: **the VM is replaceable, your work is not**. Keep your answers on your own computer, and losing the machine costs you twenty minutes instead of your homework.

Things that will cost you your VM:

- running `apt remove` or `apt upgrade` on anything
- changing the SSH port, or disabling password authentication
- closing your only terminal after changing your password, without having tested the new one

0.1 Connect and change your password

Connect with the username and address you were emailed:

```
ssh yourusername@your-vm-address
```

The image ships with a password that is not secret, so change it now:

```
passwd
```

Do not skip this step

Open a second terminal and log in with your new password before you close the first one. If the new password did not take, or your keyboard entered something other than what you thought, the still-open session is the only thing standing between you and a destroyed VM. Test first, close second.

Store the new password in a password manager. There is no recovery path.

Changing your password raises the cost of a brute-force attack, which is worth doing — but note what it does not do. These machines accept password logins from anywhere on the internet. Key-based authentication and a tool like **fail2ban** are what actually stop the attacks; a strong password only makes them slower. We will come back to access control later in the course.

0.2 Load the data

Download both databases from the course site and load them:

```
wget https://COURSE-SITE/soundwave.sql
wget https://COURSE-SITE/soundwave_small.sql
sudo mysql < soundwave.sql
sudo mysql < soundwave_small.sql
```

Both scripts drop and recreate their database, so running them again is always safe and always gives you a clean copy. That is your repair command for the rest of the semester.

Then create the account you will actually use for this assignment. It can read the data and nothing else, which means no query you write for Part 2 can damage anything:

```
sudo mysql < setup.sql
```

From here on, connect to MySQL as that user:

```
mysql -u soundwave_ro -p soundwave
```

If something breaks

Re-run the two load scripts above. That fixes any damage to the data.

If MySQL itself will not start, or you cannot log in at all, email me — do not spend an evening on it. Replacement VMs take up to one business day, which is the other reason this part is due on the 12th.

0.3 Verify

(5 points)

Run exactly this, and paste the complete output into your PDF:

```
echo "== $(whoami)@$(hostname) =="
sudo passwd -S $(whoami)
mysql -u soundwave_ro -p -t -e "SELECT VERSION();
    SELECT 'artist' t, COUNT(*) n FROM soundwave.artist
    UNION ALL SELECT 'album', COUNT(*) FROM soundwave.album
    UNION ALL SELECT 'track', COUNT(*) FROM soundwave.track;"
```

That single block proves your account works, your password has been changed (the date in the `passwd -S` output), your read-only user exists, and your data loaded. **Do not paste your password anywhere.**

0.4 Back up your work

The simplest habit that protects you: **write your queries in a text editor on your own computer**, and paste them into the terminal to test. Your answers then live somewhere the VM cannot take with it.

If you would rather edit on the VM, copy the file back to your machine at the end of every session:

```
scp yourusername@your-vm-address:~/hw1.sql .
```

Canvas accepts resubmissions, so uploading `hw1.sql` early and often costs nothing and gives you a third copy.

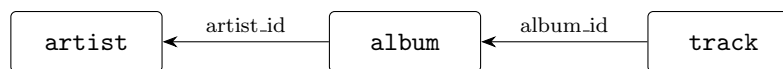
Part 1 — Relational algebra

(50 points)

Work these out by hand against the instance printed below — the same **Soundwave** database from the practice set. Do not use MySQL to find the answers; the point is to be able to reason about an expression without running it. (You will get to check three of them in Part 2.)

The schema

```
artist(artist_id, name, country, formed_year)
album(album_id, title, artist_id, release_year, genre)
track(track_id, album_id, title, duration_sec)
```



Arrows run from each foreign key to the primary key it references, and they are exactly the conditions you need when you combine relations. Two things worth remembering: **album.title** and **track.title** are different attributes that happen to share a name, so using both at once requires ρ ; and an artist reaches a track only through **artist** $\rightarrow$ **album** $\rightarrow$ **track**, so any question about both ends needs all three relations.

The instance

artist

artist_id	name	country	formed_year
A1	Neon Vale	USA	2011
A2	Sofia Marchetti	Italy	2004
A3	The Tidal	UK	2016
A4	Kwame Osei	UK	2013
A5	Hollow Pines	USA	2009
A6	Yuki Tanabe	Japan	2018
A7	Marta Reyes	Spain	2015

album

album_id	title	artist_id	release_year	genre
B1	Glass Harbor	A1	2019	Indie
B2	Second Light	A1	2022	Indie
B3	Notturmo	A2	2018	Classical
B4	Tide and Timber	A3	2021	Rock
B5	Accra Nights	A4	2020	Afrobeat
B6	Dust Chorus	A5	2017	Folk
B7	Dust Chorus II	A5	2022	Folk
B8	Kirameki	A6	2023	Electronic
B9	Cielo Bajo	A7	2021	Latin

track			
track_id	album_id	title	duration_sec
T01	B1	Harbor Lights	245
T02	B1	Saltwater	198
T03	B1	Glass	312
T04	B2	Second Light	224
T05	B2	Undertow	187
T06	B3	Notturmo I	402
T07	B3	Notturmo II	355
T08	B4	Timber	231
T09	B4	Spring Tide	265
T10	B5	Accra Nights	208
T11	B5	Highlife	243
T12	B6	Dust	279
T13	B7	Chorus	196
T14	B8	Kirameki	182
T15	B9	Cielo	254
T16	B9	Bajo	219

Notation

Use the operators from lecture: σ (selection), Π (projection), ρ (rename), $\times$ (Cartesian product), $\cup$, $\cap$, $-$, and γ (aggregation). To combine relations, take their product and put every condition in a single σ , joined with $\wedge$ — the style used in the practice set. You are not required to use a join operator.

A subscript belongs to its operator, not to the whole expression: the predicate sits under the σ , and Π , ρ and γ carry their attribute list, new name and aggregate the same way.

RA1. (4 points)

Find the titles of all tracks shorter than 200 seconds. Write the expression, then list the resulting tuples.

RA2. (5 points)

Write an expression for the distinct countries that appear in **artist**.

Then answer, in one or two sentences: how many tuples does your result contain, how many rows does **artist** have, and why do the two numbers differ?

RA3. (6 points)

Find the names of artists who have at least one track shorter than 200 seconds. Write the expression, then list the resulting tuples.

RA4. (6 points)

Find all pairs of tracks that appear on the same album, where the first track is longer than the second. Your result should have three attributes: the `album_id`, the title of the longer track, and the title of the shorter track.

Write the expression, then list the resulting tuples. (You will need ρ : both tracks come from the same relation.)

RA5. (6 points)

Find the names of artists who have never released an album in the Indie genre. Write the expression, then list the resulting tuples.

RA6. (6 points)

For each album, find the total duration of all tracks on that album. Write the expression, then list the resulting tuples.

RA7. (6 points)

A classmate wrote the following, intending to find each Folk album together with the name of the artist who released it:

$$\Pi_{\text{artist.name, album.title}} (\sigma_{\text{album.genre}=\text{"Folk"}} (\text{artist} \times \text{album}))$$

- Exactly how many tuples does this expression produce against the instance above? Show how you got the number.
- Explain in one or two sentences what is wrong with it.
- Write a correct expression, and list the tuples it produces.

RA8. (6 points)

For each pair below, say whether the two expressions are equivalent — that is, whether they produce the same result on *every* possible instance of the schema, not just this one. Justify each answer in a sentence.

(a)

$$E_1 = \Pi_{\text{title}} (\sigma_{\text{release_year} \geq 2020} (\text{album})) \quad E_2 = \sigma_{\text{release_year} \geq 2020} (\Pi_{\text{title}} (\text{album}))$$

(b)

$$E_3 = \sigma_{\text{duration_sec} > 240} (\sigma_{\text{album_id}=\text{"B1"}} (\text{track})) \quad E_4 = \sigma_{\text{album_id}=\text{"B1"}} (\sigma_{\text{duration_sec} > 240} (\text{track}))$$

If they are equivalent, state the general rule your answer relies on.

RA9.*(5 points)*

Consider:

$$\Pi_{a.name} (\sigma_{a.artist_id=b.artist_id \wedge b.release_year > a.formed_year+5} (\rho_a(\mathbf{artist}) \times \rho_b(\mathbf{album})))$$

- (a) Describe in one English sentence what this expression computes.
- (b) List the resulting tuples. Be careful at the boundary.

Part 2 — SQL

(45 points)

Now the same schema, in MySQL, on your VM.

Q1–Q9 run against the soundwave database, which holds a few hundred artists and several thousand tracks — too much to read, which is the point. **Q10–Q12 run against soundwave_small**, which is exactly the instance printed on the previous pages. Those three are translations of your own Part 1 answers, and a correct translation returns exactly the tuples you worked out by hand. That is your check on both halves at once.

Submission format

Put all twelve answers in one file called `hw1.sql`, like this:

```
-- Q1
SELECT ...;

-- Q2
SELECT ...;
```

The rules are strict because the file is graded by a script:

- Each answer is preceded by `-- Qn` alone on its own line.
- Each answer is **exactly one statement**, ending in a semicolon.
- Answers appear in order, Q1 through Q12. Do not skip numbers — if you are not answering one, leave the marker with no statement under it.
- Any other comments you want to write are fine anywhere else.

Check your file before you submit

Run the posted validator against your file:

```
python3 validate_submission.py hw1.sql
```

It tells you whether your file parses into twelve queries and whether each one runs. It does *not* tell you whether your answers are right. A file that fails the validator will lose points that have nothing to do with SQL, so run it.

How Part 2 is graded

Your query's result is compared against the reference result. So:

- **Return exactly the columns asked for, in the order given.** Column names matter only where the question names them.

- **Row order matters only where the question says to order.** Where it does, your `ORDER BY` must match.
- **Duplicate rows are compared.** If the reference returns a value once and you return it three times, that is wrong. This is the single most common way to lose points here, and it is exactly the distinction Part 1 was drilling.
- Every question can be answered with a single `SELECT`. Nothing here modifies data, and your read-only account could not do so anyway.

Questions Q1–Q9 (database: soundwave)

Q1. *(3 points)*

List the `track_id`, `title` and `duration_sec` of every track whose title contains the string `water`, in any capitalization. Order by `title`, then `track_id`.

Q2. *(3 points)*

List each distinct country appearing in `artist`, excluding artists whose country is not recorded. Order alphabetically. One column, named `country`.

Q3. *(3 points)*

In a single row, report how many rows `artist` has and how many artists have a recorded country. Two columns, named `total_rows` and `known_countries`.

Compare the two numbers with your answer to RA2. They differ for a different reason than the numbers there did — make sure you can say why.

Q4. *(4 points)*

List the `album_id` and `title` of every album whose release year is not recorded, ordered by `album_id`.

Q5. *(4 points)*

For every track appearing on an Afrobeat album by an artist from Ghana, list the artist's name, the album's title, the track's id and the track's title. Order by artist name, then album title, then track id. Four columns, named `artist_name`, `album_title`, `track_id`, `track_title`.

Q6. *(4 points)*

List the `artist_id` and `name` of every artist who has not released any album, ordered by `artist_id`. (An inner join cannot answer this question. Think about why before you write anything.)

Q7. *(4 points)*

For each artist who has released six or more albums, list the artist's name and the number of albums they have released. Order by album count descending, then name ascending. Two columns, named `name` and `album_count`.

Q8. *(4 points)*

For each genre, list the genre and the average duration of all tracks on albums of that genre, rounded to one decimal place. Order by genre. Two columns, named `genre` and `avg_duration`.

Q9. *(4 points)*

List the `track_id`, `title` and `duration_sec` of every track whose duration is equal to the longest duration found anywhere in `track`. Order by `track_id`.

Questions Q10–Q12 (database: `soundwave_small`)

Q10. *(4 points)*

Translate your RA1 expression into SQL. One column, named `title`, ordered by title. Your result should be exactly the tuples you listed in RA1.

Q11. *(4 points)*

Translate your RA3 expression into SQL. One column, named `name`, ordered by name.

Count your rows and compare with RA3. If the two disagree, do not just add something until they match — work out what Π does that `SELECT` does not, and then fix it.

Q12. *(4 points)*

Translate your RA5 expression into SQL. One column, named `name`, ordered by name.

RA5 used set difference. SQL can say this at least three ways: `EXCEPT`, `NOT IN`, or an outer join that keeps the non-matching rows. Any of the three earns full credit. If you have time, write a second one as a comment underneath and see whether they agree.

Total: 100 points (Part 0: 5, Part 1: 50, Part 2: 45), plus 5 bonus for a $\text{\LaTeX}$ -typeset Part 1.